

★ WEEK 10 · PART 2 · PRACTICE

Forward and Viterbi, Worked by Hand

Detect a Regime Switch in Real Time; Decode the Best Path

The two core HMM algorithms, run over a return sequence

Part 2's algorithms, with the arithmetic shown. We run the forward recursion over three returns to detect a regime switch as it happens, then run Viterbi on the same data to decode the single most likely regime path.

2P

PART 2 · APPLICATIONS

Sum for the Belief, Max for the Path

Case A: the forward algorithm, chaining regime beliefs over a return sequence to detect a switch in real time. Case B: Viterbi, decoding the single most likely regime path from the same data. Then a Claude Code lab.

What Part 2 Told Us

Now we run two of them - with numbers

1

Evaluation, by the forward algorithm

How likely is this data, and what regime are we in now? Chain the beliefs forward, summing over paths.

2

Decoding, by Viterbi

Which single regime sequence best explains the data? The same recursion, but keep the best path, not the sum.

3

Learning, by Baum-Welch

Fit the transition and emission parameters when the states are hidden. We flag it here and use its output.

Everything here is a calculator exercise: one HMM, three returns, two recursions.

How These Cases Work

Data, then arithmetic, then meaning

The shared setup

- The calm/turbulent HMM from Part 1.
- Three returns: +0.8, then -2.5, then -5.0.
- One calm day, then two bad ones.

What you should watch

- How the forward belief climbs toward turbulent.
- How Viterbi commits to a single path.
- How sum and max give different answers.

The Forward Algorithm

Detect the switch as it happens

Part 1 did one Bayes step; the forward algorithm chains it over a sequence. At each return, we carry every regime's belief forward through the transitions, weight it by how well that regime explains the return, and read off the filtered probability we're in each regime
- detecting a switch in real time

The recursion

- Carry each regime's forward score ahead.
- Push it through the transition matrix.
- Weight by the emission likelihood.

What we'll do by hand

- Initialize from the prior and first return.
- Step through returns two and three.
- Read the filtered turbulent probability.

The Forward Recursion

Carry every path forward, summed

$$\alpha_t(j) = \left[\sum_i \alpha_{t-1}(i) a_{ij} \right] b_j(o_t) \quad (\text{carry every path forward, summed})$$

The two moves

- Sum over where you could have come from.
- That's the transition part, in brackets.
- Then multiply by how likely this return is.

Why it's efficient

- It never enumerates full paths.
- Each step reuses the last step's scores.
- Linear in the length of the data.

Initialize, Then Step Forward

The first two forward scores

$$\begin{aligned} \text{init: } \alpha_1(C) &= 0.85(0.38) = 0.324, & \alpha_1(T) &= 0.15(0.11) = 0.017 \\ \alpha_2(T) &= [0.324(0.05) + 0.017(0.80)] \times b_T(-2.5) = 0.0296 \times 0.117 = 0.00347 \end{aligned}$$

Initialize from the prior times the first emission. Then the turbulent score at step 2 sums both ways in, weighted by how well turbulence explains the -2.5% return.

The Forward Table

Scores, and the filtered probability

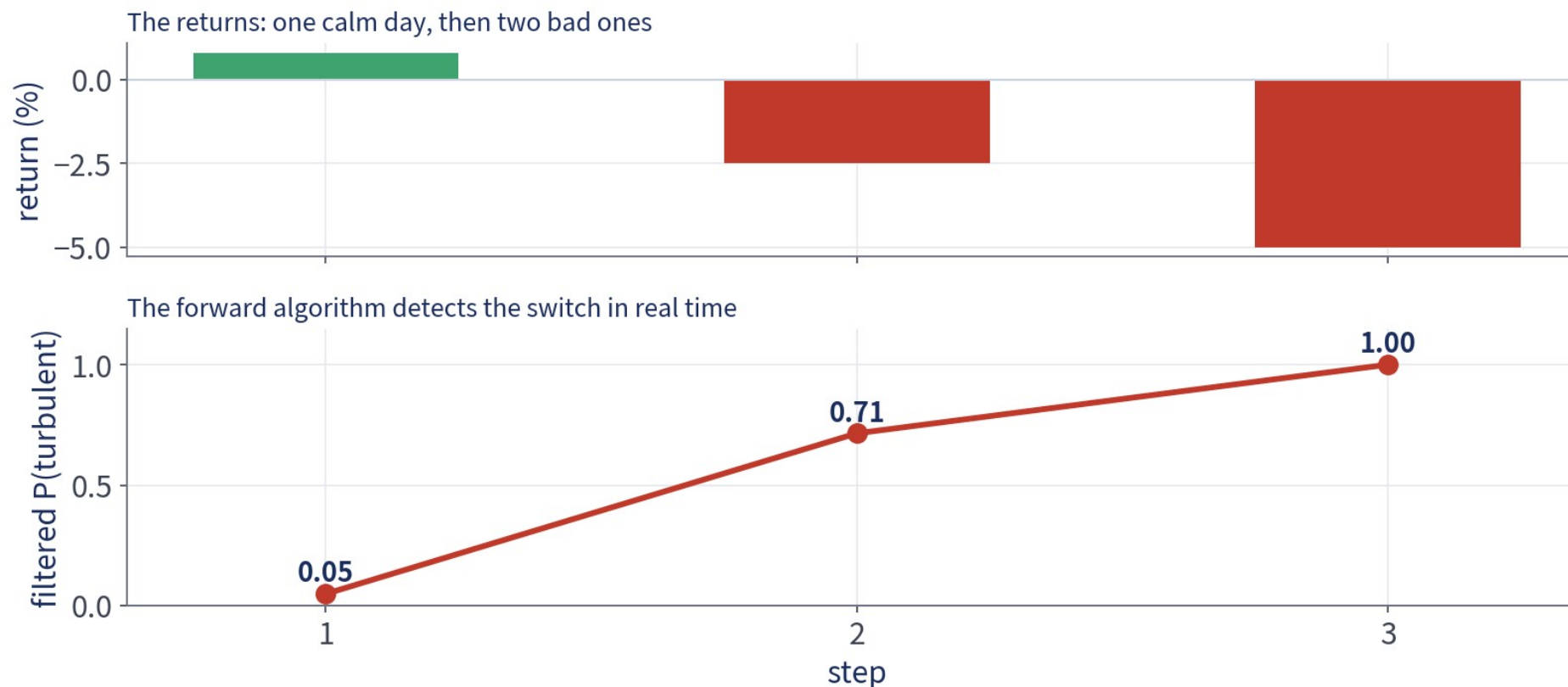
step	return	alpha (Calm)	alpha (Turb)	P(Turbulent data)
1	+0.8	0.324	0.017	0.05
2	-2.5	0.00138	0.00347	0.72
3	-5.0	~0	0.000155	1.00

Normalizing each row gives the filtered probability. It climbs 0.05 -> 0.72 -> 1.00 - the forward pass detects the regime switch as the bad returns arrive.

The Belief Climbs in Real Time

One calm day keeps the turbulent probability near 5%. The first -2.5% return pushes it past 70%; the -5.0% return pins it at essentially 100%. The forward algorithm turns a stream of returns into a live, updating read on the regime

Case A: filtered regime probability climbs 0.05 -> 0.72 -> 1.00



What the Forward Pass Gives

A live regime probability, and a likelihood

The filtered probability

- A real-time read on the current regime.
- It updates the instant a return arrives.
- This is regime filtering, discrete-state.

The likelihood byproduct

- Summing the last column gives $P(\text{data} \mid \text{model})$.
- That's how you compare or fit HMMs.
- One pass, two useful outputs.

The Viterbi Algorithm

Decode the single best regime path

The forward algorithm gives a probability at each step; Viterbi answers a different question: what one regime sequence best explains the whole series? It runs the same recursion but keeps the single best path into each state rather than summing - then backtracks to recover the labels

The difference

- Forward sums over all paths.
- Viterbi keeps only the best one.
- Max instead of sum, plus back-pointers.

What we'll do by hand

- Run the delta recursion over the returns.
- Track which prior state was best.
- Backtrack to the most likely path.

Sum for Belief, Max for the Path

One data set, two questions

forward: $\sum_i$ (all paths, soft probability) Viterbi: $\max_i$ (one path, hard labels)

Forward (sum)

- Soft probabilities at each step.
- Answers 'how likely is each regime now?'
- Marginals, one step at a time.

Viterbi (max)

- One hard sequence of labels.
- Answers 'what path best explains it all?'
- A single global decode.

The Viterbi Recursion

Keep the single best path

$$\delta_t(j) = \max_i [\delta_{t-1}(i) a_{ij}] b_j(o_t) \quad (\text{keep the single best path, maxed})$$

The recursion

- Take the max over where you came from,
- not the sum - the best predecessor only.
- Then weight by the emission, as before.

The back-pointers

- At each step, record which prior state won.
- They're the trail of breadcrumbs.
- Backtracking them recovers the path.

The Viterbi Table

Best scores and back-pointers

step	delta (Calm)	delta (Turb)	best prev -> Calm	best prev -> Turb
1	0.324	0.017	-	-
2	0.00136	0.00190	Calm	Calm
3	~0	0.0000832	Calm	Turb

At each step keep the higher-scoring way in and remember it. At step 3, the best score is turbulent, and its back-pointer leads back through turbulent then calm.

Backtrack to the Path

Follow the back-pointers home

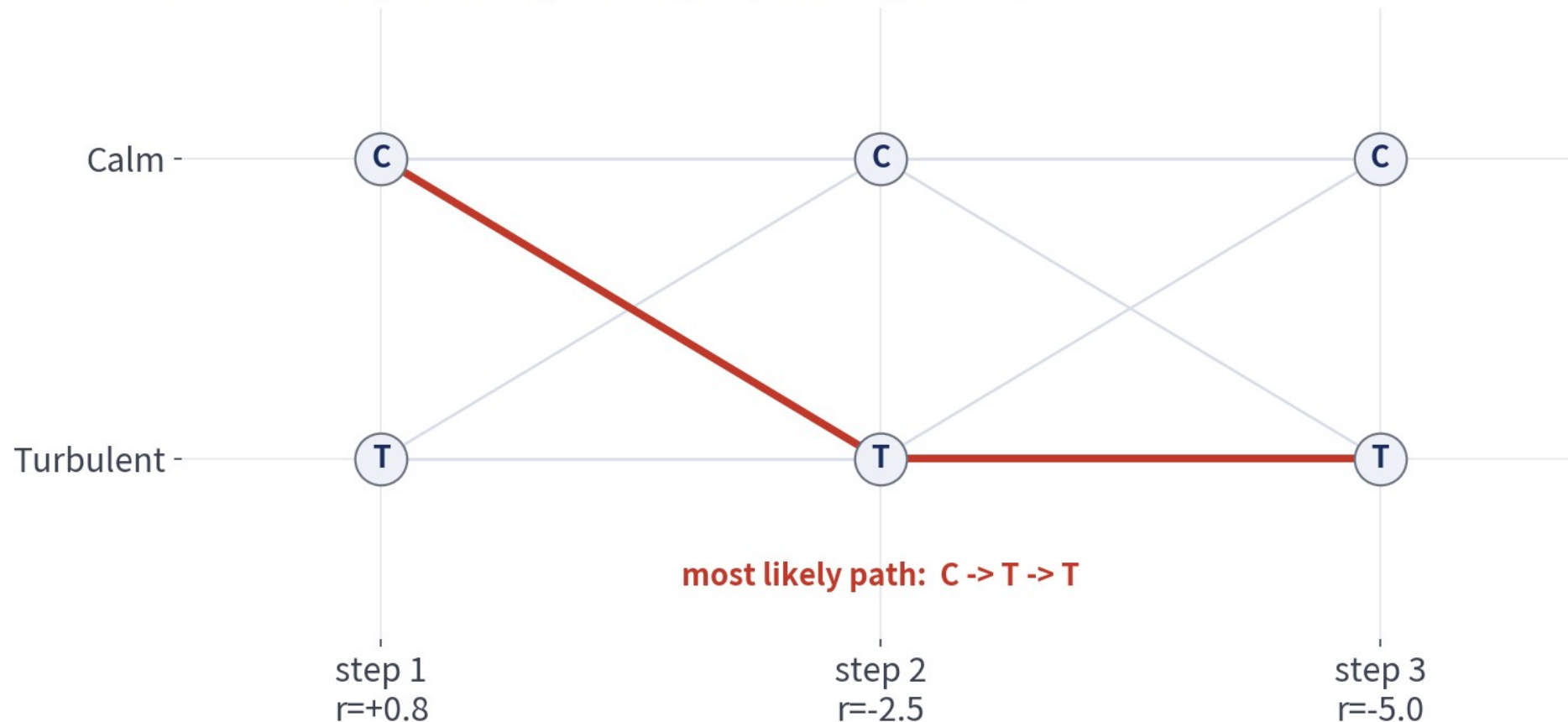
$$\text{best last state} = T \xrightarrow{\text{back-pointer}} t_2 : T \xrightarrow{\text{back-pointer}} t_1 : C \Rightarrow \text{path} = \mathbf{C}, \mathbf{T}, \mathbf{T}$$

Starting from the best final state and following the back-pointers gives Calm, Turbulent, Turbulent - the single most likely regime history for these three days.

The Best Path Through the Trellis

Every step has two possible regimes and every pairing is a possible transition. Viterbi finds the single red path - Calm, Turbulent, Turbulent - that maximizes the probability of the whole sequence, committing to one clean regime history.

Case B: Viterbi picks the single best regime path through the trellis



Why Decode a Hard Path

Labels for backtesting and attribution

When you want the path

- To label a regime history for a backtest.
- To attribute P&L to calm vs turbulent days.
- When a single clean story is what you need.

The subtlety

- The best path isn't the most likely state each step.
- Stringing per-step winners can be inconsistent.
- Viterbi keeps the whole sequence coherent.

Two Recursions, One Trellis

Sum, or max

Both algorithms sweep the same trellis with the same shape of recursion. Swap the sum for a max and you go from a soft, filtered probability to a single hard path. Learning - Baum-Welch - sits on top, using these passes to re-estimate the transition and emission parameters from data

Forward - sum

- Soft regime probabilities in real time.
- The data likelihood as a byproduct.
- Filtering, for discrete states.

Viterbi - max

- One most-likely regime path.
- Clean labels for the whole series.
- Decoding, in a single sweep.

Reproduce Forward and Viterbi

Direct Claude Code to check your arithmetic

GOAL Verify both recursions on the three returns, then run them on a real series.

[INPUT] Prompt · Claude Code

```
Write and run part2_practice.py that, for the
calm/turbulent HMM and returns [0.8, -2.5, -5.0]:
FORWARD:
  * compute alpha at each step; print the filtered
    P(turbulent) and the total data likelihood;
VITERBI:
  * compute delta and back-pointers each step;
  * backtrack and print the most likely path.
Then fit an HMM (Baum-Welch) on a real return
series and decode its regime path.
Match every printed number to the slides.
```

Check against the slides

- filtered $P(T)$: 0.05, 0.72, 1.00.
- likelihood ~ 0.000155 .
- Viterbi path: C, T, T.
- sum $\rightarrow$ soft, max $\rightarrow$ hard.

Deliverable: part2_practice.py whose printed numbers match the forward table and the Viterbi path.

Three Questions on the Cases

If you can answer these, the practice has landed

Q1 What does the forward algorithm compute?

**Filtered regime
probability + data
likelihood**

Q2 How does Viterbi differ from forward?

**It maxes over paths
instead of summing**

Q3 Why not just take the best state each step?

**The path can be
incoherent - Viterbi
keeps it whole**

All three come from the forward table, the Viterbi decode, and the sum-versus-max contrast.

Three Pitfalls to Avoid

Where the algorithms trip people

Common mistakes

- "Enumerate every path to get the likelihood."
- "Viterbi and per-step filtering must agree."
- "Forget to normalize the forward scores."

The reality

- The forward recursion avoids enumeration entirely.
- The best path can differ from per-step winners.
- Normalize to read a probability - or rescale to avoid underflow.

Two Algorithms to Remember

Part 2 applications in one screen

1

Forward detects the switch in real time

Chaining the belief over returns $[+0.8, -2.5, -5.0]$, the filtered turbulent probability climbed $0.05 \rightarrow 0.72 \rightarrow 1.00$, and summing the last scores gave the data likelihood. One pass, a live regime read plus a likelihood.

2

Viterbi decodes the single best path

The same recursion with max instead of sum, plus back-pointers, decoded the most likely regime history: Calm, Turbulent, Turbulent - a clean sequence for labeling and attribution.

3

Sum for belief, max for the path

The forward pass gives soft, filtered probabilities; Viterbi gives one hard, coherent path. Baum-Welch sits on top, using these sweeps to learn the parameters from data.

Next — Part 3 practice: regimes in action, conviction sizing, and Stage IV of the pairs strategy.